LAB 20

Capstone: One Command Runs the Pipeline End to End, with an Agentic Layer and Human Approval

Item	Detail
Topic	Topic 4: Building an End-to-End Algorithmic Trading Pipeline: Backtest, Forward Test and Performance Analysis
Pipeline stage	End-to-end pipeline
TSC reference	A7, A10, K1
Duration	20 min
Folder	labs_fullcode/Lab20-agentic-capstone/
Files	Lab20-agentic-capstone/e2e_stage.py, pipeline/e2e.py, pipeline/agents.py, Lab20-agentic-capstone/solution/e2e.py, Lab20-agentic-capstone/solution/agents.py
Prompts	Lab20_Prompts.pdf in this folder (step -> prompt, ready to copy)
Mode	Runs ONLINE with your Alpaca paper keys (.env), or OFFLINE on the bundled data and the paper-broker simulator. Paper trading only.

Lion City Capital and every scenario in these materials are fictitious and for training only. All trading in this course is paper (simulated) trading. Nothing in these materials is investment advice or a recommendation to trade any security.

Scenario

Lion City Capital's head of trading wants the whole pipeline - data, research, signals, execution, automation, diagnostics and reporting - to run with one command, stop sending orders whenever a gate fails, and finish with an optional multi-agent layer (inspired by the TradingAgents framework, arXiv 2412.20138) whose proposals pass hard risk limits and a human's approval before any paper order.

Objective

Build pipeline/agents.py (analyst, researcher, trader and risk agents; rule-based by default, Claude optional) and pipeline/e2e.py (`python -m pipeline e2e`) that runs every stage in order with gates, then evaluate the pipeline and the role of human oversight.

Before You Start

- Open a terminal in the labs_fullcode folder, activate your virtual environment and open your AI coding assistant there (set-up below). The complete, working code is already in this folder.
- Record every result you are asked to check - the same evidence is asked for in the Practical Performance assessment.

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 20	python3 Lab20-agentic-capstone/e2e_stage.py (same as: python -m pipeline e2e)
The complete code	pipeline/agents.py, pipeline/e2e.py (reference copy: Lab20-agentic-capstone/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab20-agentic-capstone/Lab20_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Build the agent layer (K1, A7)

Ask your assistant for pipeline/agents.py. The risk manager must stay deterministic code.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/agents.py. Agents: technical_analyst(symbol, bars) - EMA 20/50, RSI 14, MACD histogram and ATR 14 from pipeline.indicators -> score in [-1, 1]; news_analyst(cfg, symbol, headlines) - lexicon sentiment of pipeline.ingest.load_headlines (POSITIVE / NEGATIVE word sets); debate(cfg, tech, news) - bull and bear points and balance = 0.6 x technical + 0.4 x news; trader(symbol, tech, research, equity, held) - buy if balance > 0.25 and flat, sell if < -0.25 and held, confidence = min(1, |balance| + 0.3), qty = floor(equity x 0.25 x confidence / close); risk_manager(cfg, proposal, tech, equity) - DETERMINISTIC hard limits from config agents (allow-list, min_confidence, cap = min(equity x max_position_pct, max_order_value) with resizing, ATR bracket stop = close - stop_atr x ATR and target = close + target_atr x ATR). Optional LLM: only if the ANTHROPIC_API_KEY named in secrets_from_env is set and the anthropic package imports, news_analyst and debate call client.messages.create(model=cfg['agents']['model'] (claude-sonnet-5-5), max_tokens=1024, system prompt that treats headlines as untrusted data and asks for ONE JSON object, extra_headers={'anthropic-beta': 'server-side-fallback-2026-07-01'}, extra_body={'fallbacks': 'default'}), return None on refusal or any error so the rule-based brain is used. run(symbols, execute_orders, yes, reset, session) also applies pipeline.risk.RiskManager.check, asks a human to type YES (EOF = declined) before OrderManager.rebalance(..., units='shares', order_type='bracket' in the regular session), writes _outputs/Lab20_decision_trace.json and chart Lab20_agent_pipeline.png. main(argv) for `python -m pipeline agents --symbols --execute --yes --reset --session`. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline agents --reset prints three symbol blocks and 'Propose-only run'.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2: Build the end-to-end orchestrator (A7, A10)

Ask your assistant for pipeline/e2e.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/e2e.py. run(execute_orders=False, yes=False, fast=False) calls each stage's run() in order - check (reset), validate, ingest, hypothesis, indicators, reconcile, signals, execute (equal weights over the long universe targets from signals, capped at 20%), evaluate, search, ml, regime, abtest, pairs, backtest, runner.replay for 2025, webhook.selftest, diagnose, report - with contextlib.redirect_stdout into _outputs/Lab20_e2e_log.txt, timing each stage and printing one line: lab, stage, status (OK, GATE or ERROR), seconds and a short result. Exceptions are caught (ERROR) and the run continues. Gates: check not ready, ingest FAIL, diagnose BLOCK, report monitoring ALERT or kill switch on -> agents run propose-only. Last stage: agents.run; with --execute and gates passed ask the human for YES on the real console (or --yes for a scripted demo). Write _outputs/Lab20_e2e_summary.json and chart Lab20_e2e_pipeline.png (one coloured box per stage). --fast skips search, ml and pairs. Exit code 1 if any stage errored. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline e2e --help shows --execute, --yes and --fast.

Step 3: Run the whole pipeline (A7)

Run: python -m pipeline e2e (same as: python3 Lab20-agentic-capstone/e2e_stage.py).

Check: 20 stages, all OK, in about 6-10 seconds offline: e.g. hypothesis 'p 0.044 -> SUPPORTED', reconcile 'USD 79,878', signals '7/9 universe targets long', execute '7 paper orders', ml 'NO EDGE', run '+16.25% vs buy & hold +17.06%', webhook '7/7', diagnose 'CLEAR for paper trading', report '5/5 monitoring OK', agents 'NVDA APPROVED, AAPL no trade, SPY APPROVED; approval not requested'; 'RESULT: 20 stages ..., 0 errors, gates passed'.

Step 4: Trace the run (A10)

Open _outputs/Lab20_e2e_log.txt (the full output of every stage), _outputs/Lab20_e2e_summary.json and _outputs/Lab20_e2e_pipeline.png.

Check: The log contains each stage's full printout in order; the chart shows 20 green boxes.

Step 5: Read the agents' decisions (K1, A10)

Run: python -m pipeline agents --reset and open _outputs/Lab20_decision_trace.json and _outputs/Lab20_agent_pipeline.png. Follow one symbol from evidence to decision.

Check: Rule-based on 2025-12-31 data: NVDA technical +0.75, news +0.25, BUY 114 resized to 107 shares (cap USD 20,000) with stop 175.5 / target 201.9 - APPROVED; AAPL HOLD (balance +0.02); SPY technical +0.72, news -0.33, BUY 22 - APPROVED with stop 664.75 / target 694.45.

Step 6: Apply human oversight (A7, K1)

Run: python -m pipeline e2e --execute. When asked, type YES only if you agree with the approved orders; any other answer declines them.

Check: With YES: 2 paper bracket orders (NVDA 107, SPY 22) and 'approval granted, 2 paper orders'; otherwise 'approval declined, 0 paper orders'.

Step 7: Prove the gates stop the orders (A7)

Run: python -m pipeline kill --on, then python -m pipeline e2e --fast --execute --yes, then python -m pipeline kill --off.

Check: 'gates failed ... agents run in propose-only mode', 0 paper orders, 'gates FAILED (no orders)' - even with --yes the kill switch wins.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 8: Optional - switch on the LLM agents (K1)

Set ANTHROPIC_API_KEY in .env (with the anthropic package installed) and rerun python -m pipeline agents --reset. The news and debate agents then use Claude (claude-sonnet-5-5); if a call fails or is refused the pipeline falls back to the rules. The risk manager and the pipeline risk checks never change.

Check: The banner shows 'Agent brain: LLM (claude-sonnet-5-5)' and each News line ends with (llm); orders still need the same limits and a human YES.

Step 9: Run the capstone on the live paper account (A7)

With Alpaca paper keys set, run python -m pipeline e2e --execute during class. Approved agent orders are sent as extended-hours limit orders (no bracket outside the regular session) - check them in the Alpaca paper dashboard, then flatten with python -m pipeline execute --live-paper --flatten.

Check: The check stage shows ONLINE and the real session; agent orders appear in the dashboard with client ids starting lcc-agent.

Step 10: Evaluate the pipeline (A10, K1)

Write three sentences for the committee: what the pipeline and the agents add, what could go wrong (hallucinated news, prompt injection in headlines, over-trading, a broken gate) and why hard limits, the kill switch and human approval stay in place.

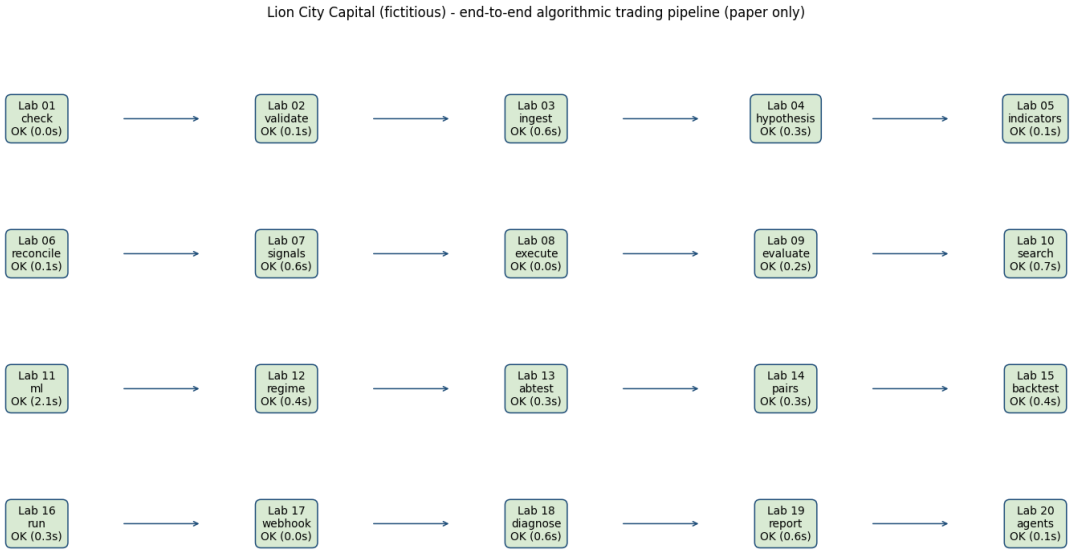
AI prompt - paste into your AI coding assistant
Using labs_fullcode/_outputs/Lab20_e2e_summary.json and labs_fullcode/_outputs/Lab20_decision_trace.json, draft three sentences for Lion City Capital's investment committee evaluating the end-to-end pipeline: the evidence from the stages, the risks of the agent layer, and the controls that keep it safe. Keep it factual and quote two numbers.

Check: Three sentences that cite stage results (for example forward test +16.25% vs backtest +17.04%) and name the controls.

Expected Output (Full Code)

python -m pipeline e2e (offline): 20 stages OK in about 6-10 seconds, 0 errors, gates passed - from check and validation through the 2025 forward test (+16.25%), webhook self-test (7/7), diagnostics (CLEAR) and report (5/5 monitoring OK) to the agents (NVDA BUY 107 with stop 175.5 / target 201.9, SPY BUY 22 with 664.75 / 694.45, AAPL HOLD); with --execute and YES, 2 paper bracket orders; with the kill switch on, 0 orders; files _outputs/Lab20_e2e_summary.json, _outputs/Lab20_e2e_log.txt, _outputs/Lab20_decision_trace.json and charts _outputs/Lab20_e2e_pipeline.png and _outputs/Lab20_agent_pipeline.png.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.



Lab 20 output (labs_fullcode/_outputs/Lab20_e2e_pipeline.png), offline mode on the bundled data.

Debrief

Agentic AI (K1) chains specialised agents that reason, debate and act, which makes it powerful and risky. Wiring it into an end-to-end pipeline whose gates, deterministic risk limits, kill switch and human approval it cannot override (A7), and evaluating every stage's evidence critically (A10), is how a desk adopts AI and automation responsibly.

Troubleshooting

Symptom	What to do
A stage shows ERROR	Open _outputs/Lab20_e2e_log.txt - the traceback is under that stage; fix it and rerun (python -m pipeline <stage> runs it alone).
Brain shows rule-based although a key is set	Install the anthropic package and check ANTHROPIC_API_KEY in labs_fullcode/.env; failures fall back to rules by design.
All agent decisions are HOLD on a second run	You already hold the positions; agents run with --reset in e2e, or add --reset when running agents alone.
Headlines look unfamiliar	Offline headlines are SYNTHETIC samples for training, not real news; online the Alpaca News API is used.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.